



* Lab Title: **Functions (Part II)**

○ Learning objectives:

The following topics will be covered in this lab session

- Function Call by Value
- Function Call by Reference
- Default values in parameters
- Random Number generation

✦ Calling Functions:

👉 Call by Value:

Call by value is used when whenever the called function does not need to modify the value of the caller's original variable. In this case a copy of the variable is passed to the function and when the function changes its values then it has no effect on the value of the variable in the main function.

• Program:

```
#include<iostream>
using namespace std;
void func( int );
int main( ) {
    int i = 8;
    func( i );
    cout<<"the value of i is"<< i<<endl;
    system("pause");
    return 0;
}
void func( int i) {
    i = i + 10;
}
```

- **Program:**

```
#include<iostream>
using namespace std;
void interchange(int,int);
int main()
{
    int x=50, y=70;
    interchange(x,y);
    cout<<"x="<<x<<" and "<<"y="<<y<<endl;
    system("pause");
    return 0;
}
void interchange(int x1,int y1)
{
    int z1; z1=x1;  x1=y1; y1=z1;
    cout<<"x1="<<x1<<" and "<<"y1="<<y1<<endl;;
}
```

- **Output:**

```
x1=70 y1=50
x=50 y=70
```

➤ **Call by Reference:**

If you are calling by reference, it means that compiler will not create a local copy of the variable which you are referencing to. It will get access to the memory where the variable saved. When you are doing any operations with a referenced variable you can change the value of the variable.

- **program:**

```
#include<iostream>
using namespace std;
void interchange(int&, int&);
int main()
{
    int x=50, y=70;
    cout<<"before calling the function the values are"<<"x="<<x<<" and
y="<<y<<endl;
    interchange(x,y);
    cout<<"after calling the function the values are"<<"x="<<x<<" and
y="<<y<<endl;
    system("pause");
    return 0;
}
void interchange(int &x1,int &y1)
{
    int z1; z1=x1; x1=y1; y1=z1;
}
```

Here the function is called by reference. In other words, address is passed by using symbol "&" and the value is accessed by using symbol "*".

The main difference between them can be seen by analysing the output of program1 and program2.

The output of Example 2 that is call by value is

```
x1=70 y1=50
x=50 y=70
```

But the output of Example 3 that is call by reference is

```
*x=70 *y=50
```

```
x=70 y=50
```

This is because in case of call by value the value is passed to function named as interchange and there the value got interchanged and got printed as

```
x1=70 y1=50
```

and again since no values are returned back and therefore original values of x and y as in main function namely

x=50 y=70 got printed.

Passing by reference is also an effective way to allow a function to return more than one value. For example, here is a function that returns the previous and next numbers of the first parameter passed.

```
// more than one returning value
#include <iostream>
using namespace std;

void prevnext (int x, int& prev, int& next)
{
    prev = x-1;
    next = x+1;
}

int main ()
{
    int x=100, y, z;
    prevnext (x, y, z);
    cout << "Previous=" << y << ", Next=" << z;
    return 0;
}
```

Output:

Previous=99, Next=101

➤ Default values in parameters:

When declaring a function, we can specify a default value for each of the last parameters. This value will be used if the corresponding argument is left blank when calling to the function. To do that, we simply have to use the assignment operator and a value for the arguments in the function declaration. If a value for that parameter is not passed when the function is called, the default value is used, but if a value is specified this default value is ignored and the passed value is used instead.

```
// default values in functions
#include <iostream>
using namespace std;

int divide (int a, int b=2)
```

```
int r;
r=a/b;
```

Output:

6

5

➤ Random Number Generation:

rand () function is used for generating random numbers. This function is defined in standard library of C. The output of the rand () function could be in the range of 0 to 32767 which is the upper limit for an integer.

```
#include<iostream>
using namespace std;
int main()
{
    int number;
    for(int i=1; i<=10; i++)
    {
        number=rand();
        cout<<number<<endl;
    }
    system("pause");
    return 0;
}
```

Lab Tasks

- Task 1: Function with Parameters

1. Declare a function called "multiply" that takes two integer parameters: "num1" and "num2".
2. Inside the function, calculate the product of the two numbers.
3. Display the result within the function.
4. Call the function from the main program, passing two numbers, to perform the multiplication.

➡ Input:

```
#include <iostream> // input/output ke liye library
using namespace std; // std namespace use kar rahe hain

// function declaration (2 integer parameters)
void multiply(int num1, int num2)
{
    int result = num1 * num2; // dono numbers ka product calculate kiya

    cout << "Multiplication = " << result << endl;
    // result screen par display kiya
}

int main()
{
    multiply(5, 4); // function call → 5 aur 4 pass kiye

    return 0;
```

The product of 4 and 5 is: 20

➡ Output:

• Task 2: Function with Return Value

1. Declare a function called "getSquare" that takes an integer parameter: "number".
2. Inside the function, calculate the square of the number.
3. Return the result.
4. Call the function from the main program, passing a number, and display the square.

↪ Input:

```
#include <iostream> // input/output library
using namespace std; // standard namespace

// function declaration (integer return karega)
int getSquare(int number)
{
    int square = number * number; // number ka square
    calculate kiya

    return square; // result wapas main function ko return
    kiya
}

int main()
{
    int result; // variable result store karne ke liye

    result = getSquare(7); // function call → 7 pass kiya

    cout << "Square = " << result << endl;
    // result display kiya

    return 0; // program end
}
```

↪ Output:

The Square of 7 is 49

Conclusion

In this lab, we learned different types of functions in C++ and how parameters are passed using call by value and call by reference. We also implemented functions with return values and default parameters. These concepts are essential for writing efficient and modular programs. Call by reference is useful when we need to modify original data, while call by value is safer when we want to protect original variables.